SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Duration: 60 Mins **QUESTIONS: 5** **Total Marks:50**

Annexure A

Experiments

Code of Conduct:

I MONISH L [1972260035] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student



RUBRICS FOR CALCULATION

Criteria	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Marks/10
Weightage	**25**	**30**	**20**	**15**	**10**	**100%**
Q1						
Q2						
Q3						
Q4						
Q5						

Signature of the Faculty *Total Marks Awarded:*

Annexure A

Experiments

QUESTIONS:4 **Total Marks:40**

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)

   Answer: TCP establishes a reliable connection using a three-step process called the three-way handshake. First, the client sends a SYN (synchronize) segment to the server with an initial sequence number, signaling a request to open a connection. Second, the server replies with a SYN-ACK segment, which sets its own initial sequence number and acknowledges the client's sequence number (ACK = client seq + 1), showing the server is willing to establish the connection. Third, the client sends an ACK segment acknowledging the server's sequence number, completing the handshake. Each flag has a distinct role: SYN synchronizes sequence numbers so both sides can track data in order, and ACK confirms receipt of the other side's sequence number. This exchange lets both hosts agree on starting sequence numbers, confirm each other's willingness and readiness to communicate, and detect duplicate or delayed connection requests from previous sessions, which is what makes the subsequent data transfer reliable and ordered.

2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)

   Answer: When a DNS query is captured over UDP in Wireshark, the packet shows a very compact structure: an 8-byte UDP header (source port, destination port, length, checksum) directly followed by the DNS payload. In contrast, a TCP header is at least 20 bytes and carries additional fields such as sequence number, acknowledgment number, window size, and control flags, none of which are present in UDP. Because UDP has no sequence numbers or acknowledgments, it does not track whether a datagram arrived, does not retransmit lost packets, and does not

enforce ordering. This lightweight, connectionless design removes the overhead of connection setup and state tracking, so a DNS query can be sent and answered in a single round trip, giving very low latency. The trade-off is reduced reliability: if a query or response is lost, the resolver itself must handle retries and timeouts, which is acceptable for short, idempotent requests like DNS lookups but would be unsuitable for larger, ordered data transfers.

3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)

   Answer: When packet loss is simulated (for example, by briefly disconnecting the network), Wireshark captures show clearly different behavior for TCP and UDP. For TCP, the sender notices missing acknowledgments or duplicate ACKs and retransmits the unacknowledged segments after a retransmission timeout or via fast retransmit, and the receiver reassembles the stream in the correct order once the missing segment arrives. This can be seen in Wireshark as "TCP Retransmission" or "TCP Dup ACK" flagged packets. UDP, on the other hand, has no built-in mechanism to detect or recover lost datagrams; if a UDP packet is dropped during the disconnection window, it is simply gone, and no retransmission packet appears in the capture. This difference exists because TCP maintains sequence numbers, acknowledgments, and timers specifically to guarantee delivery and order, which requires extra round trips and buffering, making it slower but reliable. UDP skips all of this control overhead, making it faster and more efficient for loss-tolerant, latency-sensitive traffic such as streaming or DNS, at the cost of reliability.

4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP. (BL4) (10)

   Answer: Capturing DNS traffic while resolving a domain name in Wireshark shows a query packet (DNS opcode QUERY) containing a question section with the requested name and query type, such as A (IPv4 address), AAAA (IPv6 address), or MX (mail exchange record). The corresponding response packet contains the same

transaction ID and an answer section with the resolved record(s), or an error code if resolution failed. The resolution time can be measured directly in Wireshark using the time delta between the query and response packets, which is typically only a few milliseconds for a cached or nearby resolver, and higher for uncached or geographically distant queries. DNS predominantly uses UDP because queries and responses are small, single-round-trip exchanges where the overhead of TCP's three-way handshake and connection teardown would add unnecessary latency for very little benefit; UDP's lightweight, connectionless nature keeps typical lookups fast, and DNS falls back to TCP only when responses exceed the UDP payload size or for operations such as zone transfers.